

Owner Android App V1 — Final Project Audit

AK VisionFlow Owner Android App · Phases 1–4 (Foundation, Dashboard, Alerts & Push, AI Advisor)

Originally prepared September 16, 2026 · Re-verified and updated September 18, 2026

Update note (September 18, 2026): after this audit was originally written, the backend went through a separate architecture-hardening program (its own Phase 0.1–0.8) that touched most of the rest of the application. As part of that program, the Owner Android app's backend counterpart was found to have been quietly **paused** — its routes unmounted and its database migration never declared in the schema, so the app could not have reached the backend at all in that state. This has since been **fully re-verified compatible and restored to a locally testable state** (see the new Section C.1 and the refreshed evidence throughout this report). Per the client's explicit instruction, this restoration is deliberately **not activated for real tenants or devices** — it makes the app testable again, it does not put it live. Sections below are updated in place; nothing from the original findings was removed, only re-verified and, where the evidence changed, corrected.

A. Scope and Method

This audit covers Phases 1–4 of the AK VisionFlow Owner Android App as implemented against the four Phase specification documents, re-verified with fresh evidence on September 18, 2026. Evidence gathered:

- Fresh full backend Jest run (real, disposable local Postgres, never production), including — for the first time — the 5 Owner Mobile backend test files: **415/415 tests passed, 18/18 suites, exit 0**. (The original audit's 332/332 figure could not include the mobile suite at all, because the backend could not run it in the paused state described above.)
- Fresh full Android `assembleDebug + testDebugUnitTest + lintDebug`, forced clean (not a cached result): **BUILD SUCCESSFUL — 58/58 unit tests passed, 0 lint issues, a real installable APK produced**. This is the first time this exact build has been executed in this project's environment; the original September 16 audit was written before this evidence existed.
- Direct code inspection of every mobile backend route, the JWT/auth isolation layer, the database migration, and Android security-relevant files (token storage, network config, manifest, build config) — re-confirmed unchanged from the original audit.
- A read-only production database status check (no write/deploy/reset ever executed against production, then or now).

B. Git / Deployment State

All Owner Android work remains local-only. Nothing has ever been pushed to the remote repository, either at the time of the original audit or now. Do not treat any part of this report as evidence of production deployment.

C. Production Database State

Verified via a read-only status check against the production database: the schema migration adding push notification and alert-preference tables has **still not been applied to production** — unchanged since the original audit. The mobile backend cannot function against production until this migration is deployed there — an action requiring the client's explicit authorization, not performed as part of this audit or its update.

C.1 What changed between the original audit and this update

Between September 16 and September 17, an unrelated backend architecture program found that the Owner Mobile backend routes had been unmounted and its database migration had never been declared in the ORM schema file — a state in which the app's backend could not have served a single request, even in a local test environment. On September 18, this was investigated and restored:

- **Schema/database drift closed:** the migration's tables (device tokens, notification preferences, push configuration) had, it turned out, already been silently created in every local test database this whole time — but the ORM's schema file never declared them, so the backend code could not actually use them. The schema file has been corrected to match what the migration already creates. This is a declarative fix only; it changes nothing about what data exists, and was applied to local test databases only, never to production.
- **Routes re-connected locally:** the six Owner Mobile API routes were re-connected in the local codebase (not deployed anywhere) so they could be tested again, together with a dedicated login rate-limiter matching the same protection the staff and customer-portal logins already have.
- **A genuine integration gap found and fixed:** when the app was originally paused, the scheduled job that is supposed to dispatch push notifications and daily summaries had been disconnected from the rest of the automation pipeline and never reconnected. This was found via the restored test suite (5 tests failed until this was fixed) and has been repaired — this is a real bug fix, not a re-verification.

None of this activates the app for any real business or device. It restores the app to the same "coded, tested, not yet deployed" state the original audit described — now with materially stronger evidence behind that claim than existed on September 16.

D. Cross-Phase Integration

Traced end-to-end: Auth → Home → Analytics → Alerts → AI Advisor → Profile → Logout, and the KPI → alert → AI → push → deep-link pipeline. No duplicated business logic (Alerts and AI Advisor read the same underlying data through two independent, non-mutating display mappings). No broken navigation, no stale-cache logic gaps, no API version mismatches found. Date/tenant/branch handling is consistent and correctly isolated per tenant's own timezone.

E. Phase 1 — Security Audit

Check	Result
Non-GET requests blocked at the API level, not just the UI	Verified
Staff / mobile / portal token mutual exclusivity	Verified — tested with a real cross-surface token rejection test
Only the business owner role can obtain/use a mobile token	Verified — a staff (non-owner) login is rejected even with the correct password
Deactivation takes effect immediately, not just at token expiry	Verified
HTTPS enforced in production builds	Verified — cleartext HTTP is permitted only in local development builds, never in release builds
Session token stored securely on-device	Verified (code-level) — Android Keystore-backed encrypted storage, not plain preferences
Versioned API surface	Verified — isolated from both the staff web API and the customer portal API

No blocking issues.

F. Phase 2 — Data / Analytics Audit

The Android app performs no independent recalculation — every figure shown is rendered directly from the same, already-tested backend service the main web application uses. Filter/comparison logic and empty/error/loading states are present and correct. No blocking issues.

G. Phase 3 — Alerts / Push Audit

A real cross-tenant penetration test confirms a second business account receives "not found," never another business's alert data. Marking an alert read/dismissed was verified in a real test to leave the underlying business record (e.g. product stock) completely unchanged. Notification preferences, minimum-priority filtering, and device-token registration are all correctly scoped per tenant. No blocking issues.

H. Phase 4 — AI Advisor Audit

A real cross-tenant test confirms a second business account sees zero data derived from another tenant. Every figure shown by the AI Advisor traces directly to existing, already-tested analytics — nothing is invented or fabricated. The daily brief and trend-forecast caching is enforced at the database level, preventing any duplicate record even under a rapid double-tap of "refresh."

Assessment of the deterministic (non-LLM) AI approach

The system uses rules/statistics-based analysis rather than a generative AI chatbot. This is assessed as an **acceptable approach for this version** — the specification's completion criteria (daily brief, anomaly flags, trend direction, evidence-backed recommendations) are all satisfied without requiring a conversational AI, and the app is explicitly decision-support (data + suggested action), never an automated decision-maker. The two previously-disclosed depth limitations (brief detail level; expense-trend evidence on quiet periods) are both classified as **acceptable, deliberate design choices**, not defects requiring a fix.

I. AI Safety / Grounding, Data Integrity, and Multi-Tenant/RBAC Review

No fabricated data anywhere in the evidence path. No cross-tenant or cross-branch leakage. No AI-triggered business-data mutation exists (every AI route is read-only). The database migration adding new tables/columns for this feature set is purely additive — no destructive statements. A code-and-real-test-level penetration review (cross-tenant object access, foreign branch IDs, unauthorized roles, tampered/expired tokens, malformed requests, direct object ID guessing) found every boundary correctly enforced. **PASS** on all four areas.

J. Existing Web System Regression

The full pre-existing backend test suite — covering products, customers, suppliers, categories, branches, sales (including reversal), purchases and receiving, payments, optical orders, stock/inventory, expenses, the existing AI Recommendation Center, and (as of the unrelated architecture program between the original audit and this update) multi-company/branch/warehouse management and a centralized permissions system — passed alongside all mobile-specific tests, with **zero regressions** introduced by this work or by restoring Owner Mobile compatibility.

K. Android Build/Quality Audit

Build, unit tests, and lint all pass cleanly — re-confirmed September 18 with a forced-clean rebuild (not a cached result) in an environment that, unlike September 16, has a working Android SDK installed: **58 of 58 unit tests passed, lint reported "No issues found," and a real installable debug APK was produced.** No hardcoded secrets or API keys were found anywhere in the app. Network request logging is limited to method/URL/status only — the session token is never written to any log, and all logging is disabled entirely in release builds. App permissions are minimal (internet access only). Session token storage uses hardware-backed encryption.

L. Production Readiness

Item	Status
Backend logic, tenant isolation, RBAC, AI safety	Verified locally (real automated tests, re-confirmed September 18 at 415/415)
Android build / unit tests / lint	Verified locally (real, forced-clean build run, re-confirmed September 18 at 58/58)
Backend/database declarations match what the app needs	Corrected September 18 (was previously out of sync — see Section C.1); local test databases only
Code pushed to the shared repository	Still not done
Database schema present in production	Still not applied — requires client authorization
Production API address configured in a release build	Still not configured — must be set at release-build time
Real push notification credentials (vs. the local test provider)	Still not set up — requires a paid account action by the client
On-device or emulator verification	Still not possible in any environment available to this project (see Addendum)

Automated tests passing is not the same as being production-ready. The same client-owned actions identified in the original audit still remain before this app can go live: publishing the code, applying the pending database update to production, and configuring the production server address and push credentials in a release build. This update makes the underlying evidence for "the code works" stronger and more current; it does not shorten this remaining list.

M. Backup / Recovery

The client's production database backup/point-in-time-recovery capability was **not tested** — doing so safely requires creating a disposable copy of the database via the hosting provider's own tools, which is a client action, not something performed automatically during this audit (to avoid any risk to live data).

N. Verdict

🟡 **GO WITH CONDITIONS** — unchanged from the original September 16 verdict, now backed by fresher and more complete evidence (415/415 backend tests including the mobile suite itself, and a real forced-clean Android build). The application code, security architecture, tenant isolation, and automated test coverage for Phases 1–4 are sound and verified. The remaining conditions are the same deployment/infrastructure actions as before

(publishing the code, applying the production database update, configuring production settings, real push credentials) — not code defects — plus on-device verification, which two separate attempts across two machines have still not been able to complete for environmental reasons outside this audit's control.

Addendum — Manual On-Device Testing Attempt

Following the original audit, a genuine attempt was made to perform manual, on-device testing of the Android app (login, navigation, and all Phase 1–4 screens) rather than relying on build success alone, as good practice requires.

What was done (September 16): An Android emulator and system image were installed, a local test backend and database were set up with real sample data, and a virtual test device was created.

What blocked completion (September 16): The emulator could not start because hardware virtualization (AMD-V / SVM) appeared disabled at the BIOS/firmware level on the machine used for that attempt — a setting that can only be changed by physically entering the machine's BIOS setup, outside the scope of a software audit.

A second attempt was made September 18 (on the environment used for that day's re-verification, which by then had a working Android SDK, unlike September 16). The existing virtual test device started but failed at a later stage, reporting: *"x86_64 emulation currently requires hardware acceleration — Android Emulator hypervisor driver is not installed on this machine."* This is a related but distinct failure point from the September 16 attempt (a missing hypervisor driver rather than a BIOS setting), consistent with virtualization support being unavailable in some form on both machines tried so far.

Current status: on-device/manual testing remains NOT VERIFIED, across two separate attempts on two different points of failure. This is disclosed honestly rather than assumed or substituted with build/test-suite success. Once hardware virtualization is enabled and the emulator's hypervisor driver is installed (or a physical Android device is made available), the full manual smoke test (login/logout, Home, Analytics, Alerts, AI Advisor screens, navigation, loading/empty/error states) can be completed and this report updated accordingly.